

From SDE to Elite AI Engineer

*A complete, step-by-step roadmap with 7 phases, 7 real projects,
and a 90-day execution plan. No math overload. No fluff.*

7	7	90	0
Phases	Real Projects	Day Plan	Math Derivations

Phase 1: Absolute Basics (Wk 1–2) · Phase 2: Prompt Engineering (Wk 3–4) · Phase 3: Embeddings & Search (Wk 5–7) ·
Phase 4: RAG Systems (Wk 8–11) · Phase 5: Agents (Wk 12–16) · Phase 6: Fine-Tuning (Wk 17–21) · Phase 7:
Production (Wk 22–26)

01

Absolute Basics — LLMs & Your First API Calls

No prior AI knowledge assumed. Start here.

Goal: Understand what an LLM actually is (without the math), make your first API calls, understand tokens and costs, and build a working chat program. Finish this phase comfortable calling LLM APIs and knowing what's happening underneath.

CONCEPTS TO LEARN

What is an LLM? — A language model trained on massive text to predict the next token. Not magic — a large function that takes text in and produces text out. No reasoning, no memory between calls.

Tokens — the atomic unit — LLMs don't see words — they see tokens. ~1 token ≈ 0.75 English words. 'ChatGPT' = 1 token. Every API call costs per input token + per output token. Always measure costs.

Context window — The LLM can only see the last N tokens. GPT-4o: 128k tokens (~96k words). Everything outside this window is invisible. This is a hard engineering constraint.

Temperature & sampling — Temperature 0 = deterministic. Temperature 1 = creative/random. Use 0–0.3 for production, 0.7–1 for creative tasks.

System vs User prompt — System prompt = your instructions/persona (you set). User prompt = what the user types. The LLM sees both every call.

Model tradeoffs — GPT-4o: best quality, most expensive. GPT-4o-mini: fast, cheap, surprisingly capable. Claude 3.5: best for long context. Know quality vs cost vs speed.

TOOLS FOR THIS PHASE

openai	Python SDK for OpenAI API
anthropic	Python SDK for Claude API
tiktoken	Count tokens before sending — manage costs
python-dotenv	Load API keys from .env safely
rich	Pretty terminal output for your CLI chatbot

PHASE 1 PROJECT

Multi-Model CLI Chatbot with Cost Tracking

Build a terminal-based chatbot that maintains conversation history across turns, supports both OpenAI and Claude (user can choose), displays token count and running cost after every message, and lets the user change temperature mid-conversation.

STEP BY STEP

1. Load API keys from .env. Set up both OpenAI and Anthropic clients.
2. Maintain a `messages[]` list — append user and assistant turns to it.
3. After every response, print: tokens used, estimated cost, model used.
4. Add `/model` and `/temp` commands to switch model or temperature mid-chat.
5. Add `/clear` to reset conversation, `/save` to dump history to a .txt file.

Skills: [OpenAI SDK](#) · [Anthropic SDK](#) · [tiktoken](#) · [Conversation History](#) · [Cost Tracking](#)

This proves: *You understand the chat completions API, conversation state, token economics, and multi-provider patterns — the foundation of everything that follows.*

02

Prompt Engineering & Structured Output

Learn to speak to LLMs precisely. Get reliable, structured responses.

Goal: Stop getting unpredictable LLM responses. Master techniques to reliably control format and reasoning quality. By the end, you can build an AI feature that behaves consistently in production.

CONCEPTS TO LEARN

Zero-shot vs Few-shot — Zero-shot: just describe the task. Few-shot: give 2–5 input/output examples. Few-shot dramatically improves consistency. Use real examples from your data, not toy ones.

Chain of Thought (CoT) — Add 'Think step by step' before the answer. Forces the model to show its work. Dramatically improves accuracy on complex tasks. More tokens, but worth it.

System prompt engineering — Structure: [Persona] → [Task] → [Constraints] → [Output format] → [Negative examples]. Be explicit about what NOT to do.

Function / Tool calling — Define functions the model can call. It returns structured JSON with the function name and arguments. This is how agents work at their core.

Structured output with Pydantic — Use the Instructor library to guarantee a Pydantic model as output. If the LLM returns malformed JSON, Instructor auto-retries. Game-changing for production.

Prompt versioning — Treat prompts like code. Version them. A 5-word change can cause a 30% accuracy swing. Never change production prompts without A/B testing.

TOOLS FOR THIS PHASE

instructor	Guaranteed structured Pydantic output from any LLM
pydantic	Data validation + schema definition
tenacity	Retry logic for API failures
fastapi	Start wrapping AI calls in a real API endpoint

PHASE 2 PROJECT

AI Content Pipeline with Structured JSON Output

FastAPI service that accepts raw text (job description, product review) and returns a validated JSON breakdown using Instructor + Pydantic. Extend: batch endpoint with asyncio, basic prompt versioning system.

STEP BY STEP

1. Define a Pydantic model for your output (e.g., JobPosting with 6+ fields).
2. FastAPI POST endpoint: accepts raw text, returns the Pydantic model as JSON.
3. Add /batch endpoint — accept list of texts, process with asyncio.gather().
4. Build prompt versioning: load from prompts/v1.txt, prompts/v2.txt, etc.
5. Add /compare endpoint: run 2 prompt versions on same input, show both outputs.

Skills: FastAPI · Instructor · Pydantic · Async Python · Prompt Versioning

This proves: Production-ready AI API with guaranteed output schemas, async processing, and disciplined prompt management.

03

Embeddings & Semantic Search

Teach your app to understand meaning, not just keywords.

Goal: Understand how text gets converted into numbers that encode meaning, store and search them efficiently, and build a semantic search system that finds related content even when exact keywords don't match.

CONCEPTS TO LEARN

What is an embedding? — A dense vector (~1536 floats) representing the meaning of text. Similar text → geometrically close vectors. 'Dog' and 'Puppy' are closer than 'Dog' and 'Database'.

Cosine similarity — The standard way to compare vectors. Returns -1 to 1 (1 = identical meaning). You don't need to code it — numpy and vector DBs handle it.

Embedding models — Use text-embedding-3-small (fast, cheap) or text-embedding-3-large (better). Open-source: BGE-M3 (multilingual, near-OpenAI quality, free). Always use the same model for indexing and querying.

HNSW — how vector search works — Exact nearest-neighbor search in 1536-dim space is impossibly slow. HNSW builds a hierarchical graph for approximate nearest neighbors in milliseconds. All vector DBs use this internally.

Metadata filtering — Always store metadata (source, date, category) alongside vectors. Real queries combine semantic + metadata filter. 'Find similar to X but only from last 30 days.'

When embeddings fail — They miss exact matches (IDs, codes, proper names). Combine with BM25 keyword search — this is hybrid search. Use Reciprocal Rank Fusion (RRF) to merge results.

TOOLS FOR THIS PHASE

qdrant-client	Vector DB — open-source, fast, excellent Python client
openai embeddings	text-embedding-3-small/large API
rank-bm25	BM25 keyword search for hybrid retrieval
sentence-transformers	Free local embedding models (BGE, E5, MiniLM)
numpy	Cosine similarity and vector math when needed

PHASE 3 PROJECT

Personal Knowledge Semantic Search Engine

Search engine over your notes, bookmarks, or any text corpus. Ingest 200+ items, embed each one, store in Qdrant with metadata. FastAPI search endpoint returning top-5 semantic matches. Upgrade to hybrid search.

STEP BY STEP

1. Prepare 200+ text items (notes, Notion export, bookmark descriptions).
2. Ingestion script: embed each with text-embedding-3-small, store in Qdrant + metadata.
3. Build /search endpoint: embed query, Qdrant ANN search, return top-5 with scores.
4. Add metadata filtering: /search?query=X&tag;=python&after;=2024-01-01
5. Hybrid search: run BM25 in parallel, merge with RRF. Compare quality before/after.

Skills: Qdrant · Embeddings · BM25 Hybrid · Metadata Filtering · Ingestion Pipeline

This proves: You understand the embedding + vector DB + search pipeline end-to-end — the core of every RAG system you'll ever build.

04

Retrieval-Augmented Generation (RAG)

The #1 most in-demand AI engineering skill in industry right now.

Goal: Build a complete, evaluated RAG pipeline from scratch. Master document ingestion, chunking, retrieval, reranking, generation, and evaluation. Build a Document Q&A; chatbot that cites sources and is measurably good.

CONCEPTS TO LEARN

Why RAG exists — LLMs don't know your private data. RAG injects real-time, relevant context into the prompt at query time. Solves hallucination for factual questions. The most widely deployed AI pattern.

Chunking strategy — Split documents into 256–512 token chunks with 10–20% overlap. Fixed-size, recursive, or semantic chunking — each has tradeoffs. Chunk size is one of the highest-impact tuning decisions.

The full RAG pipeline — Ingest: chunk → embed → store in vector DB. Query: embed → ANN search → retrieve top-K → build prompt → LLM generates answer → return with citations.

Re-ranking — ANN retrieval gets ~20 approximate matches. A cross-encoder re-ranker scores each precisely against your query and returns top-5. Dramatically improves precision.

HyDE (Hypothetical Document Embeddings) — Ask the LLM to write a hypothetical answer, then embed that. A hypothetical answer is semantically closer to actual document chunks than a raw question is.

RAGAS evaluation — Faithfulness (answer grounded in context?), Context Precision (retrieved chunks relevant?), Context Recall (all relevant chunks found?). Run after every change.

TOOLS FOR THIS PHASE

unstructured	Extract text from PDFs, DOCXs, HTML — handles messy real docs
langchain / llama-index	RAG pipeline orchestration — productivity tools
cohere	Best managed re-ranker API (Cohere Rerank)
ragas	Automated RAG evaluation — faithfulness, precision, recall
langfuse	Trace every RAG call — see exactly what chunks were retrieved

PHASE 4 PROJECT

Production Document Q&A; with Measured Quality

Full RAG system over a real document set. Multi-turn conversation, source citations with page numbers, measured RAGAS quality score, Langfuse observability, simple frontend.

STEP BY STEP

1. Ingestion: Unstructured → recursive chunking → embed → Qdrant + metadata.
2. Query: HyDE expansion → hybrid search → Cohere re-rank top-20 → top-5 chunks.
3. Generation: stuff chunks + history into prompt → GPT-4o → answer with citations.
4. Add Langfuse tracing: log retrieved chunks, latency, tokens, cost per query.
5. 50 golden Q&A; pairs. RAGAS eval target: Context Precision > 0.75, Faithfulness > 0.85.
6. Add a Streamlit or React frontend with source citation display.

Skills: Full RAG Pipeline · HyDE · Cohere Rerank · RAGAS · Langfuse · Citations

This proves: Architecture and evaluation of a production-grade RAG system — the most requested AI skill in job postings today.

05

AI Agents — Tool Use, Planning & Memory

Build systems that take actions, not just return answers.

Goal: Build AI systems that reason about what steps to take, call external tools, maintain memory across steps, and complete multi-step tasks autonomously. Understand when agents are appropriate vs overkill.

CONCEPTS TO LEARN

The agent loop — Perceive (input + memory) → Think (LLM decides) → Act (call tool) → Observe (read result) → repeat until done. Everything else is a variation of this loop.

ReAct pattern — Reason + Act interleaved. The LLM explicitly writes its reasoning ('I need to search for X because...') before deciding which tool to call. Makes behavior interpretable.

Tool definition — Define tools as Python functions with clear docstrings and typed arguments. The LLM reads the docstring to decide when/how to call each tool. Good tool descriptions are engineering work.

Memory architectures — Short-term: context window. Long-term: summaries + key facts in a DB or vector store. Episodic: full history retrieved on demand. Know which to use when.

Multi-agent systems — Orchestrator plans and delegates. Specialists (Researcher, Writer, Critic) execute. LangGraph manages state transitions. Use for complex workflows where steps are dynamic.

Safety — critical — Always: max_iterations, timeouts, output validation, sandboxed code execution, human-in-the-loop before write/delete operations. Agents can cause real damage.

TOOLS FOR THIS PHASE

langgraph	Stateful agent workflows — best for complex multi-agent systems
openai tool_use	Native function calling — the primitive all agents are built on
tavily	LLM-optimized web search API — best search tool for agents
e2b	Sandboxed code execution for agents that write and run code
mem0	Persistent memory layer for agents across sessions

PHASE 5 PROJECT

Deep Research Agent with Web + RAG

Research agent that plans its strategy, searches the web (Tavily), queries your Phase 4 RAG for internal docs, synthesizes findings, and produces a structured report with citations. Full LangGraph state machine with streaming progress.

STEP BY STEP

1. Define tools: web_search, rag_search, write_section, finish_report.
2. LangGraph state machine: PLAN → RESEARCH (loop) → SYNTHESIZE → REVIEW → DONE.
3. Critic node: after synthesis, LLM reviews draft and flags gaps. Loop back if needed.
4. Safety: max_iterations=10, timeout=60s, log every tool call to Langfuse.
5. Stream progress via SSE: 'Searching web...', 'Querying docs...', 'Writing section 2...'

Skills: LangGraph · Tool Use · ReAct · Multi-Agent · Streaming · Safety Guards

This proves: Multi-agent orchestration, complex state management, real tool integration, streaming UX.

06

Fine-Tuning Open-Source Models

When prompting + RAG isn't enough — own the model.

Goal: Understand when and why to fine-tune. Learn to curate a quality dataset, fine-tune Llama/Mistral with QLoRA, evaluate the result rigorously, and serve it in production with vLLM.

CONCEPTS TO LEARN

When to fine-tune — Fine-tune for: consistent output style/format, domain vocabulary, or when prompting + RAG hits a quality ceiling. DO NOT fine-tune to add knowledge — use RAG for that.

LoRA / QLoRA — Instead of updating all 7B parameters, LoRA adds small adapter matrices. Updates only 0.1–1% of params. Same quality gain, 10x less GPU memory. QLoRA quantizes base model to 4-bit first.

Dataset curation — Quality over quantity. 500 excellent instruction-response pairs beat 50,000 mediocre ones. Every example should represent exactly the behavior you want. Bad data = bad model.

Evaluation before and after — Compare fine-tuned vs base + good prompt on your eval set. If fine-tuned doesn't win, don't use it. Fine-tuning often hurts general capability while improving specific tasks.

vLLM for serving — PagedAttention for GPU KV-cache management. Enables efficient concurrent serving. 24x throughput vs naive HuggingFace inference. OpenAI-compatible API.

PEFT / Adapter merging — Train multiple LoRA adapters for different tasks. At inference, load base model once, swap adapters per request. Efficient multi-task serving.

TOOLS FOR THIS PHASE

unsloth	2–5x faster fine-tuning on consumer GPU — best starting point
axolotl	Battle-tested fine-tuning framework for serious training runs
vllm	Production LLM serving — OpenAI-compatible, GPU-optimized
wandb	Training metrics, loss curves, experiment tracking
RunPod	Rent GPU by the hour — A100 40GB ~\$1.50/hr

PHASE 6 PROJECT

Fine-Tuned Domain Specialist Model

Fine-tune Llama 3.1 8B with QLoRA on a real domain task (SQL gen, code review, legal extraction).
Rigorous comparison: Base + Prompt vs Fine-tuned on quality, latency, and cost.

STEP BY STEP

1. Define your task. Create 100-example eval set first — before any training.
2. Curate 500–2000 instruction-response pairs. Clean, consistent, representative.
3. Fine-tune Llama 3.1 8B with Unsloth + QLoRA on RunPod. Track loss with W&B.;
4. Evaluate: base + best prompt vs fine-tuned on 100 eval examples using LLM-as-judge.
5. Deploy with vLLM. Benchmark: throughput (req/s), latency (P50/P95), cost/1M tokens.
6. Write a blog post with result tables and methodology. Publish on Medium/Substack.

Skills: QLoRA · Unsloth · Dataset Curation · vLLM · LLM-as-Judge · Benchmarking

This proves: End-to-end fine-tuning pipeline, dataset engineering, rigorous evaluation. Blog post = powerful portfolio piece.

07

Production AI Systems — Scale, Reliability & Cost

Make it work for real users, at real scale, without breaking.

Goal: Make everything production-grade — observable, cost-efficient, resilient, secure, and scalable to 100,000+ users. This is where your SDE background is your superpower over ML researchers.

CONCEPTS TO LEARN

Cost engineering — Model routing: cheap classifier decides GPT-4o vs GPT-4o-mini per query. Semantic caching: serve cached responses for similar queries. Together: 60–80% cost reduction.

LLM observability — Every call must be traced: input, output, retrieved chunks, latency, token count, cost, model version. Langfuse for this. You can't debug AI without traces.

Resilience patterns — Circuit breaker: stop calling a failing LLM, serve fallback. Fallback chain: GPT-4o → Claude → GPT-4o-mini. Retry with exponential backoff. Rate limit queuing.

Security — Prompt injection defense. PII detection with Presidio. Output filtering with NeMo Guardrails. Never trust user input inside system prompts.

Multi-tenant AI — Namespace vectors by org_id. Row-level security in Postgres. Per-org token budgets. Per-org model routing. Audit logs. Enterprise requires this.

Deployment & scaling — FastAPI on Kubernetes with HPA. Celery + Redis for async AI jobs. Modal or RunPod for bursty GPU workloads. Separate GPU nodes for self-hosted models.

TOOLS FOR THIS PHASE

litellm	Single interface to all LLM providers, built-in fallbacks
gptcache	Semantic caching for LLM responses — 40–70% cost reduction
presidio	PII detection + anonymization library
celery + redis	Async task queue for long-running AI jobs
kubernetes	Orchestrate, scale, health-check your AI microservices

PHASE 7 CAPSTONE PROJECT

Production Multi-Tenant AI SaaS Backend

Assemble everything: multi-tenant API with JWT auth, per-org quotas, RAG + agent endpoints, model routing, semantic caching, PII redaction, circuit breaker, full observability, async queue, K8s deployment. Write a complete runbook.

STEP BY STEP

1. FastAPI + JWT auth + per-org rate limits (Redis) + multi-tenant Qdrant namespacing.
2. LiteLLM gateway with model routing (complexity classifier → cheap/expensive), fallback chain, semantic cache.
3. PII redaction middleware (Presidio) on all inputs. Audit log every request.
4. Celery + Redis for async jobs: document ingestion, batch processing, fine-tune triggers.
5. Full Langfuse integration: per-org cost dashboard, alert on cost threshold breach.
6. Kubernetes: FastAPI + Celery deployments, Redis + Qdrant StatefulSets, HPA for API pods.
7. Write a full runbook: onboard new org, rotate API keys, handle LLM outage, scale for traffic spike.

Skills: Multi-tenant · LiteLLM · Semantic Cache · PII Redaction · K8s + HPA · Runbook

This proves: You can design, build, and operate a production AI system end-to-end. Senior/staff-level work. Put this on your resume with architecture diagram and live demo URL.

RAG vs Fine-Tuning vs Prompting — Decision Framework

One of the most common AI engineering interview questions. Here's the complete framework.

Dynamic / real-time data	✗	✓✓	✗	✓✓
Private / proprietary data	✗	✓✓	■ baked in	✓✓
Consistent output style	■	■	✓✓	✓✓
Reduces hallucination	✗	✓✓	■	✓✓
Citeable sources	✗	✓✓	✗	✓✓
Cost at high scale	High	Medium	Low	Medium
Complexity	Low	Medium	High	Very High
Time to first result	Minutes	Days	Weeks	Weeks+

Decision rule (memorize this): Always start with prompting → if quality ceiling hit, add RAG → if still insufficient style/format consistency, consider fine-tuning. Fine-tuning ≠ teaching the model new facts. For new knowledge, RAG is almost always the right answer.

90-Day Execution Plan

60% building, 40% learning. Never learn without building. Never build without measuring.

Week	Learn	Build
MONTH 1 — Foundations		
Week 1–2		
Week 3–4	CoT, few-shot, system prompts, function calling, Instructor, Pydantic, async Python	Phase 2 Project: Structured content pipeline API with prompt versioning
MONTH 2 — Retrieval & RAG		
Week 5–6		
Week 7–9	Full RAG pipeline, chunking strategies, HyDE, re-ranking, RAGAS evaluation, Langfuse	Phase 4 Project: Document Q&A; with citations, RAGAS score > 0.80
MONTH 3 — Advanced		
Week 10–11		
Week 12–13	Fine-tuning criteria, LoRA/QLoRA, dataset curation, Unsloth, vLLM, evaluation	Phase 6 Project: Fine-tune Llama 3.1 8B, publish comparison blog post
Week 14–15	Cost engineering, model routing, caching, PII, circuit breakers, K8s, multi-tenant	Phase 7 Capstone: Production multi-tenant AI backend with runbook
Week 16	Interview prep: 5 AI system design problems, DSA review, portfolio polish	GitHub READMEs + architecture diagrams + 2 demo videos. Apply to 20 roles.

What SDEs Get Wrong Entering AI

✗ Studying math before building Spending months on backprop and attention derivations before writing a single line of LLM code.	✓ FIX Build first. The math doesn't help you build better RAG systems. Phase 1 takes 2 weeks, not 2 months.
✗ Fine-tuning as the first resort Reaching for fine-tuning for every domain problem before trying prompting or RAG.	✓ FIX Prompting → RAG → Fine-tuning is the correct order. Fine-tuning is the last tool, not the first.
✗ No evaluation strategy Shipping AI features without any measurement. 'It seems better' is not a quality signal.	✓ FIX Build 50 golden examples before the feature. RAGAS or LLM-as-judge on every change.
✗ Treating prompts as casual strings Hardcoding prompts inline, changing them without testing, no version history.	✓ FIX Prompts are code. Version control them. A 10-word change can cause a 30% accuracy swing.
✗ Agents for everything Using a multi-step agent loop when a simple deterministic pipeline would work reliably.	✓ FIX If steps are known at design time: pipeline. Agents are for genuinely dynamic workflows only.
✗ Zero observability No tracing, no cost tracking. AI failures are invisible. You can't debug what you can't see.	✓ FIX Add Langfuse from day 1. Log every input, output, chunk, latency, token count, cost.

The Only List You Need

BOOKS

AI Engineering — Chip Huyen (2024) — O'Reilly. THE book for production AI engineering. Read cover-to-cover.

Designing Machine Learning Systems — Chip Huyen — MLOps fundamentals. Chapters 1–6 most relevant for AI engineers.

COURSES (FREE)

DeepLearning.AI Short Courses — deeplearning.ai. 'RAG Agents', 'Fine-Tuning LLMs', 'LangChain'. Start here.

Full Stack LLM Bootcamp (Berkeley) — YouTube. Best production-focused LLM course available.

Fast.ai Practical Deep Learning — fastai.com. Best coder's path to understanding model internals.

BLOGS & NEWSLETTERS

chip.huyen.com — Best thinking on production AI systems. Every post is must-read.

eugeneyan.com — **Eugene Yan** — RAG, LLMs in production. Practical patterns from an Amazon Principal Engineer.

Ahead of AI — **Sebastian Raschka** — Best newsletter for fine-tuning and open-source model news.

GITHUB REPOS TO STUDY

awesome-llm-apps — Curated real LLM app examples with code. Project inspiration.

langfuse/langfuse — Your best observability stack. Read the architecture.

vllm-project/vllm — Production LLM serving internals. Read the architecture docs.

BerriAI/litellm — Multi-provider LLM gateway. Essential in production.

*The top 1% AI Engineers are not the ones who know the most math.
They are the ones who ship production systems that work reliably at scale.*

SDE → Elite AI Engineer Roadmap · 7 Phases · 7 Projects · 90 Days · 2025 Edition